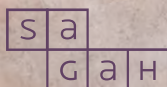


PENSAMENTO COMPUTACIONAL

Patrícia Fernanda da Silva



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Comando condicional `if` (estruturas aninhadas) — Python

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Discutir a utilização de comandos condicionais aninhados.
- Realizar testes de mesa em problemas que utilizem comandos condicionais aninhados.
- Praticar os comandos condicionais aninhados em problemas computacionais.

Introdução

Ao escrever um código, é possível determinar algumas condições, que serão executadas conforme o comando. Se a condição for verdadeira, uma estrutura será seguida; se for falsa, outra estrutura será seguida. Os comandos condicionais aninhados permitem que caminhos sejam organizados dentro de uma mesma programação; a partir deles, as decisões serão executadas.

Neste capítulo, você vai estudar sobre os comandos condicionais aninhados, verificando também como realizar testes de mesa em problemas que utilizem comandos condicionais aninhados e como praticar esses comandos em problemas computacionais.

Comandos condicionais aninhados e sua utilização

No seu dia a dia, provavelmente, algumas decisões são tomadas com base em determinadas condições. Por exemplo: se um aluno leva 30 minutos para chegar até a universidade, e a aula começa às 8 horas, esse aluno precisa tomar a decisão sobre qual é a melhor hora para sair de casa, para que chegue pontualmente na universidade. Um exemplo mais simples é: se o aluno está cansado, então, ele vai dormir. Trata-se de condições simples que devem ser observadas para que uma decisão seja tomada. Isso também ocorre durante a programação, na qual se utilizam ações e tomadas de decisão. Para tanto, são utilizados os **comandos condicionais** — códigos que, quando incluídos na programação, dependem de uma ou mais condições para que sejam executados.

Em programação, a escolha entre duas ou mais opções ocorre quando o programa lê os dados de um arquivo e realiza uma ação, tomando como base os dados lidos ou a entrada de um usuário. Essa escolha, no ambiente Python, é escrita a partir de uma instrução `if`, pela qual a condição é avaliada. Caso a condição seja verdadeira, serão realizadas as instruções apontadas como “então”. Se não for verdadeira, verificam-se as instruções após a instrução `if`, conforme leciona Lee (2011).

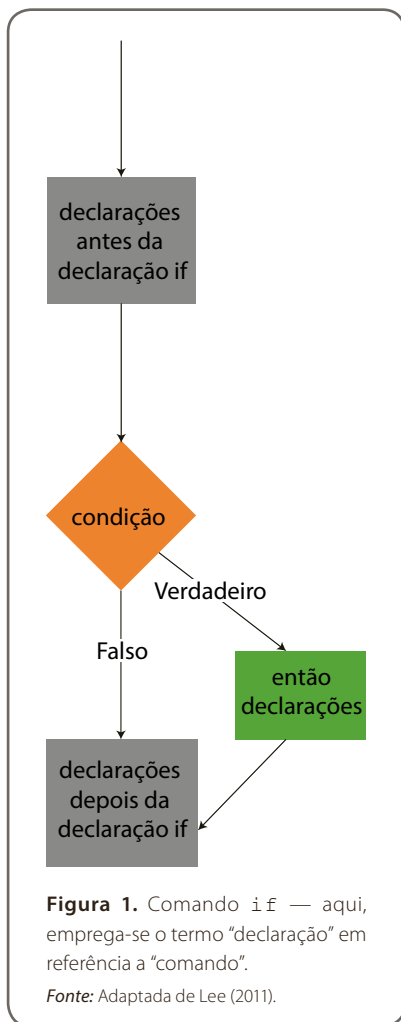


Fique atento

Ao escrever os comandos no ambiente de programação Python, observa-se que a indentação deve ser rigorosamente seguida, pois, ao contrário de outras linguagens de programação, o ambiente Python define e articula os blocos a partir dos recuos que são reconhecidos. Sendo assim, a indentação não pode ser ignorada.

De maneira análoga, pode-se imaginar a seguinte situação: ao comprar algo em uma loja, o comprador pode escolher se o pagamento vai ser realizado em cartão de crédito ou de débito ou em dinheiro. Se a compra vai ser paga em cartão, o comprador vai para um lado da fila; se vai ser paga em dinheiro, o comprador vai para o outro lado da fila. Caso o cartão seja recusado, a compra deverá ser paga em dinheiro, devendo o comprador se dirigir para a fila do dinheiro. Após o pagamento, quem pagou com cartão e quem pagou em dinheiro sairão com suas compras pelo mesmo lugar.

A Figura 1 exemplifica o uso do comando `if`, em que a condição será avaliada para dar sequência às próximas instruções de “então”.



Caso a condição seja verdadeira, será seguido um comando; caso seja falsa, outro comando será executado. Nas situações em que um comando só é realizado caso a condição seja verdadeira, após realizar a constatação, o programa executa o comando; se a condição for falsa, e o programa não for

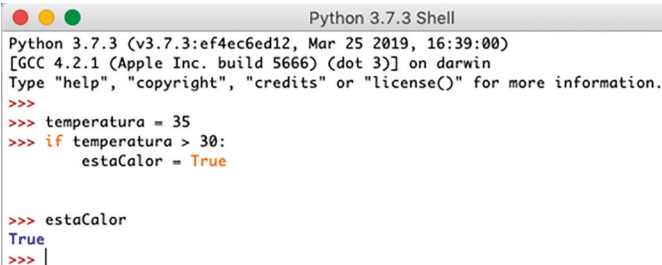
instruído com nenhum comando para a condição falsa, ele vai prosseguir para os próximos comandos que estão sem indentação.

Comando condicional `if`

O comando `if` é um comando condicional importante, que pode ser utilizado para a execução de códigos que comparem dois valores. Esse comando pode ser utilizado para as condições `sim/não` ou `verdadeiro/falso`. Ao ser utilizado, o comando `if` apresentará a estrutura representada na Figura 2.

```
if condição:
    comandoV1
    comandoV2
```

Figura 2. Representação genérica do comando `if`.



```
Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>>
>>> temperatura = 35
>>> if temperatura > 30:
    estaCalor = True

>>> estaCalor
True
>>> |
```

Figura 3. Exemplo de comando `if` utilizando Python.

O código da Figura 3 pode ser descrito da seguinte forma: se a temperatura for maior do que 30°, a variável `estaCalor` será verdadeira (`True`). Observa-se que o comando foi executado, pois a variável `estaCalor` foi apontada como `True`. Isso porque a temperatura é de 35°, ou seja, maior do que 30°.

Comando condicional `else`

Em programação, pode-se determinar algo que tenha duas respostas, isto é, duas instruções: uma para o caso de a condição ser verdadeira e outra para o caso de a condição ser falsa. O comando condicional `else` possibilita fazer alguma coisa quando a condição é avaliada como falsa. Nesse caso, a resposta poderá ser não, falso ou, até mesmo, 0.

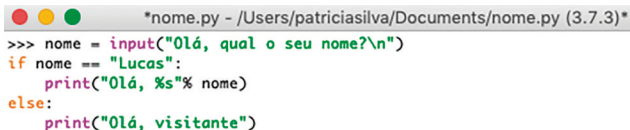
Ao utilizar os comandos `if` e `else`, a condição é avaliada. Se verdadeira, os comandos `if` são executados; se for falsa, os comandos `else` são executados (Figura 4). Os demais comandos serão executados a partir do momento que nenhuma indentação for percebida.

```
if condição:
    comandoV1
    comandoV2

else:
    comandoF1
    comandoF2
```

Figura 4. Representação genérica dos comandos `if/else`.

Pode-se ter um ou mais comandos, desde que seja observada a mesma indentação. Caso a condição não seja verdadeira, os comandos `else` passam a ser executados. Observe o exemplo da Figura 5, em que é utilizado o ambiente Python.



```
*nome.py - /Users/patriciasilva/Documents/nome.py (3.7.3)*
>>> nome = input("Olá, qual o seu nome?\n")
if nome == "Lucas":
    print("Olá, %s" % nome)
else:
    print("Olá, visitante")
```

Figura 5. Exemplo de comando `if/else` utilizando Python.

No exemplo apresentado na Figura 5, é solicitado ao usuário digitar o nome. Se o nome digitado for “Lucas”, a resposta será “*Olá, Lucas*”; se for digitado outro nome, a resposta será “*Olá, visitante*”.

Comando condicional `elif`

Caso se deseje testar mais de uma condição, será necessário utilizar o comando `elif`. Ele pode ser utilizado inúmeras vezes, para testar quantas condições forem necessárias. Caso nenhuma delas seja verdadeira, o bloco de comandos `else` será executado (Figura 6).

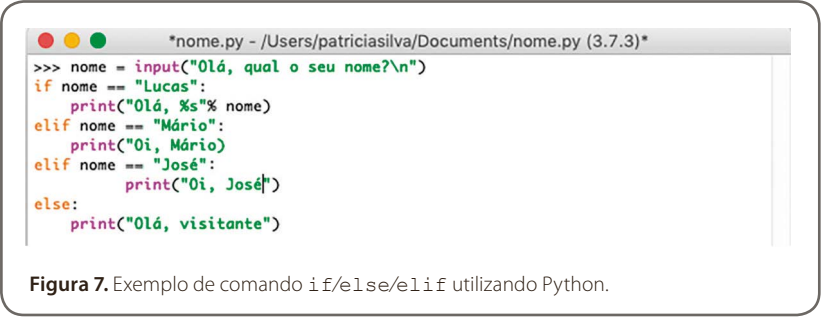
```
if condição1:
    comandoV1
    comandoV2

elif condição2:
    comandoV3
elif condição3:
    comandoV4

else:
    comandoF1
    comandoF2
```

Figura 6. Representação genérica dos comandos `if/else/elif`.

Utilizando a condição `elif`, novos comandos podem ser listados e atendidos, observando-se a indentação. Se nenhuma condição for verdadeira, os comandos `else` passam a ser executados. Observe o exemplo da Figura 7, em que é utilizado o ambiente Python.



```
*nome.py - /Users/patriciasilva/Documents/nome.py (3.7.3)*
>>> nome = input("Olá, qual o seu nome?\n")
if nome == "Lucas":
    print("Olá, %s" % nome)
elif nome == "Mário":
    print("Oi, Mário")
elif nome == "José":
    print("Oi, José")
else:
    print("Olá, visitante")
```

Figura 7. Exemplo de comando `if/else/elif` utilizando Python.

No exemplo apresentado na Figura 7, o programa solicita o nome do usuário; se for digitado Lucas, a resposta será “Olá, Lucas”; se for digitado “Mário” ou “José”, aparecerá “Oi, Mário” ou “Oi, José”. Mas, caso não seja digitado nenhum desses nomes, o comando `else` será executado, aparecendo na tela a mensagem “Olá, visitante”.

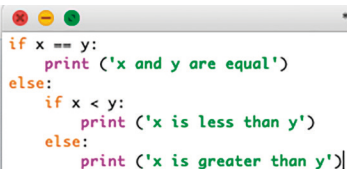


Fique atento

As variáveis que possibilitam as respostas “verdadeiro” e “falso” também são chamadas de booleanas; isso porque foram criadas por George Boole, um matemático inglês que viveu por volta de 1800 e criou a álgebra booleana.

Comandos condicionais aninhados

Ainda na temática dos comandos condicionais, Downey (2016) ressalta os **comandos condicionais aninhados**, que podem ser aninhados um dentro do outro, conforme mostra o exemplo da Figura 8.



```
if x == y:
    print ('x and y are equal')
else:
    if x < y:
        print ('x is less than y')
    else:
        print ('x is greater than y')
```

Figura 8. Exemplo de comandos condicionais aninhados um dentro do outro.

Fonte: Adaptada de Downey (2016).

Conforme exemplificado na Figura 8, que utiliza o ambiente de programação Python, a condicional exterior possui dois ramos: o primeiro traz uma instrução simples, e o segundo possui uma nova instrução `if`, que, por sua vez, possui dois ramos próprios com instruções simples, que também podem ser condicionais.

Por meio das indentações, é possível verificar as instruções evidenciadas pela estrutura das condicionais, embora sejam difíceis de se ler rapidamente. Em geral, programadores acham incômodo utilizar códigos com tantos "degraus", como na Figura 8, que são até mesmo considerados feios. Além disso, dependendo do recuo, o código pode sair do lugar, ocasionando erros na interpretação por parte do Python. A Figura 9 traz um exemplo de simplificação desses comandos.



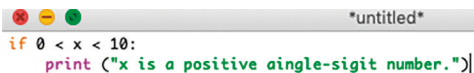
```
if 0 < x:
    if x < 10:
        print('x is a positive single-digit number.')
```

Figura 9. Exemplo de comandos condicionais aninhados simplificados.

Fonte: Adaptada de Downey (2016).

Na Figura 9, utilizando o ambiente Python, a instrução `print` é executada somente se colocada depois das condicionais. Cabe ressaltar a importância de se atentar às indentações ao escrever o programa.

O programa descrito acima ainda pode ser reescrito de forma mais concisa, conforme exemplificado na Figura 10.

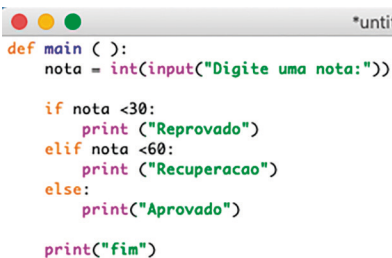


```
if 0 < x < 10:
    print("x is a positive aingle-sigit number.")
```

Figura 10. Exemplo de condição mais concisa.

Fonte: Adaptada de Downey (2016).

Além do cuidado com os "degraus" de códigos, a partir da escrita mais concisa, pode-se também utilizar outra instrução para a condição `if`: a condição `if-elif`. Esta permite que os códigos fiquem mais bem organizados, de forma que apenas uma alternativa seja escolhida. Nesse caso, se a primeira condição é interpretada como `True`, o código é executado, e as demais alternativas são ignoradas. A condição `else` pode ou não aparecer, é opcional. Veja a exemplificação na Figura 11.



```
def main():
    nota = int(input("Digite uma nota:"))

    if nota < 30:
        print("Reprovado")
    elif nota < 60:
        print("Recuperacao")
    else:
        print("Aprovado")

    print("fim")
```

Figura 11. Exemplo de condição com uso do `if/elif/else`.

Na ilustração acima, é possível perceber que a instrução `elif` se mostra como uma contração do `else-if`, permitindo que a escrita fique mais clara e objetiva para verificar as várias alternativas apresentadas. O `elif` pode ser repetido diversas vezes, podendo-se, ainda, criar categorias para alunos que tenham atingido médias superiores a 90 ou 95, por exemplo.

Teste de mesa com comandos condicionais aninhados

Para solucionar um problema, é necessário verificar se o programa é capaz de executar o que foi determinado pelo programador. Para isso, é realizada uma simulação de comandos, utilizando uma técnica conhecida como **teste de mesa**. Por meio do teste de mesa, os comandos são simulados, mas sem o uso do computador (compilador e interpretador). O teste é realizado apenas por meio de simulações, empregando o uso de papel e caneta, de modo que o futuro programador possa prever como a programação no ambiente Python será realizada.

Em grande parte das linguagens de programação, assim como no Python, o programa necessita tomar decisões a partir de valores e resultados gerados no decorrer da sua execução. Ou seja, o programa decide se uma instrução específica será ou não executada, conforme a sua condição. No caso do teste de mesa dos comandos condicionais `if`, `else` e `elif`, esses comandos serão testados para verificar se realmente a programação elaborada pelo programador pode ser executada pelo programa.

Vejamos como se pode executar o teste das instruções `if`, `else` e `elif`. Para tanto, vamos criar um código que calcule a média de um aluno em um processo seletivo. A média para aprovação no teste é 5, e será selecionado o aluno que obtiver a maior média após as três provas realizadas.

Diante dessas informações, deve-se primeiramente pensar e elaborar a escrita desse código usando apenas papel e caneta. Depois, pode-se tentar criá-lo no ambiente de programação Python e verificar a sua execução. Ao registrar as notas dos alunos (nota 1, nota 2 e nota 3), será necessário o uso do comando `float` (ponto flutuante), pois as notas podem ser números decimais — nesses casos, esse comando deve ser utilizado. A Figura 12 exemplifica o código para o programa pretendido.

```
*untitled*  
nota1 = float(input('Primeira nota: '))  
nota2 = float(input('Segunda nota: '))  
nota3 = float(input('Terceira nota: '))  
  
media = (nota1 + nota2 + nota3) / 3  
  
print('Media: ', media)  
  
if media < 5.0:  
    print('Reprovado')  
elif media < 10:  
    print('Aprovado')  
else:  
    print('Aprovado com Distinção!')
```

Figura 12. Código para o cálculo da média dos alunos, conforme o problema proposto.



Fique atento

`True` e `False` são valores especiais e pertencem ao tipo `bool`. Os operadores `==` (dois sinais de igualdade) são relacionais — um dos sinais de igualdade é de atribuição. Downey (2016) aponta outros exemplos de operadores:

```
x != y    # x não é igual a y  
x > y     # x é maior do que y  
x < y     # x é menor do que y  
x >= y    # x é maior ou igual a y  
x <= y    # x é menor ou igual a y
```

Resolvendo problemas com comandos condicionais aninhados

Antes de resolver os problemas com o uso do ambiente de programação Python, é importante que você possa pensar de forma lógica sobre os comandos a serem utilizados e a forma como serão ordenados, para posteriormente serem compilados e executados. Para tanto, antes de tentar resolver os problemas que serão propostos, sugere-se que você escreva os códigos no papel ou, até mesmo, no bloco de notas do seu computador, para posteriormente escrevê-los no ambiente de programação Python.



Link

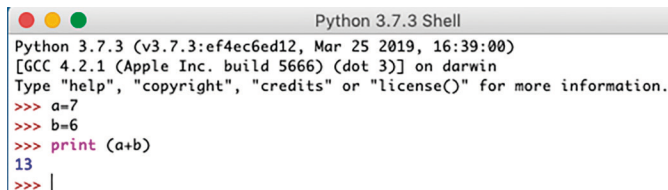
Acessando o *link* a seguir, você pode visitar a página Python Progressivo, na qual estão disponíveis diversas opções de exercícios e explicações complementares sobre a linguagem de programação Python, com diferentes graus de dificuldade.

<https://qrgo.page.link/BMSTF>

A seguir, serão demonstrados exemplos de uso do ambiente Python para a resolução de problemas. É importante salientar que um mesmo problema pode ter diferentes formas de resolução. Aqui, serão priorizadas as respostas mais objetivas.

Para dar início à resolução de problemas, temos o seguinte enunciado: se a soma de a e b é 13, e sabe-se que o valor de a é 7, qual é o valor de b ? Como descrever que a soma de $a + b = 13$? Demonstre por meio dos comandos Python.

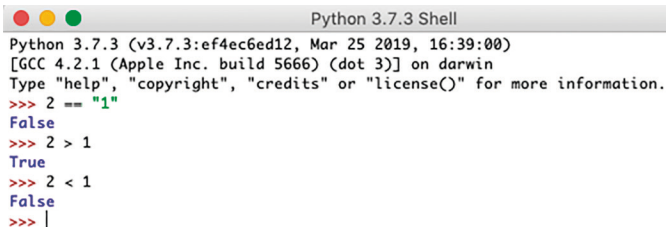
Ao pensar no problema acima, primeiro temos que definir o valor de b , para, posteriormente, podermos escrever a adição utilizando o comando `print`, conforme mostra a Figura 13.



```
Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>> a=7
>>> b=6
>>> print (a+b)
13
>>> |
```

Figura 13. Resposta para o problema proposto.

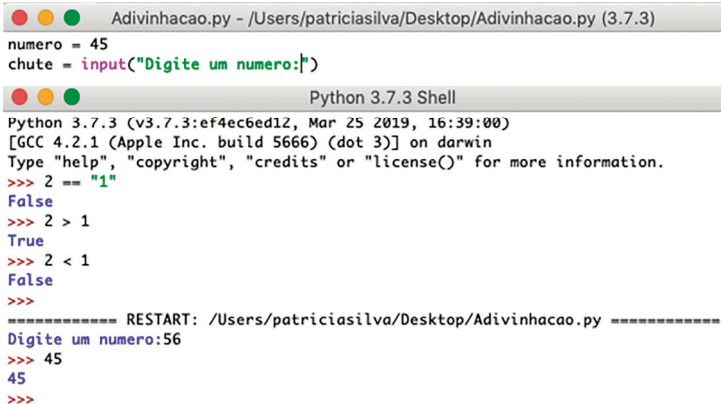
Em situações-problema, também se pode utilizar os valores booleanos (`True` e `False`) para verificar expressões. Acompanhe o exemplo da Figura 14.



```
Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2 == "1"
False
>>> 2 > 1
True
>>> 2 < 1
False
>>> |
```

Figura 14. Exploração de valores booleanos.

O exemplo da figura 15 traz os comandos utilizados quando um número é digitado. Após a execução da programação, é avaliado se o número digitado é o mesmo do escrito no código.

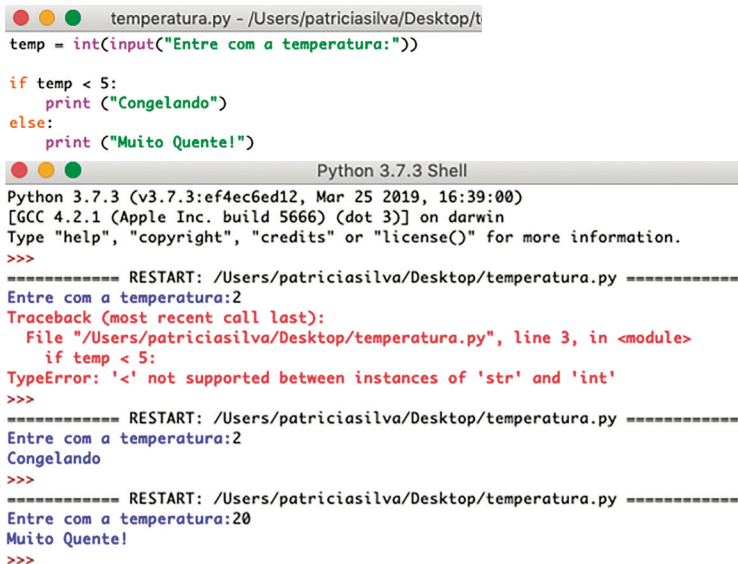


```
Adivinhacao.py - /Users/patriciasilva/Desktop/Adivinhacao.py (3.7.3)
numero = 45
chute = input("Digite um numero:")

Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2 == "1"
False
>>> 2 > 1
True
>>> 2 < 1
False
>>>
===== RESTART: /Users/patriciasilva/Desktop/Adivinhacao.py =====
Digite um numero:56
>>> 45
45
>>>
```

Figura 15. Jogo de adivinhação.

Os comandos `if-else` permitem o uso de mais condições, que podem ser verdadeiras ou falsas, conforme visto anteriormente. Na Figura 16, os comandos `if-else` são utilizados para analisar a condição da temperatura.



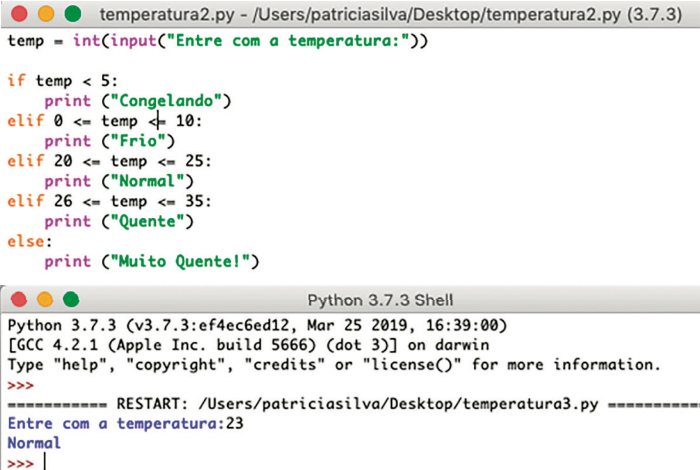
```
temperatura.py - /Users/patriciasilva/Desktop/t
temp = int(input("Entre com a temperatura:"))

if temp < 5:
    print ("Congelando")
else:
    print ("Muito Quente!")

Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /Users/patriciasilva/Desktop/temperatura.py =====
Entre com a temperatura:2
Traceback (most recent call last):
  File "/Users/patriciasilva/Desktop/temperatura.py", line 3, in <module>
    if temp < 5:
TypeError: '<' not supported between instances of 'str' and 'int'
>>>
===== RESTART: /Users/patriciasilva/Desktop/temperatura.py =====
Entre com a temperatura:2
Congelando
>>>
===== RESTART: /Users/patriciasilva/Desktop/temperatura.py =====
Entre com a temperatura:20
Muito Quente!
>>>
```

Figura 16. Programação com uso dos comandos `if/else`.

Nessa programação, observa-se o uso de uma variável, que receberá um valor. Se menor do que 5, existe uma condição verdadeira; caso contrário, se maior do que 5, a condição é falsa. Entre `if` e `else`, ainda seria possível colocar mais algumas condições, com situações de temperaturas intermediárias, conforme mostra a Figura 17.



```
temperatura2.py - /Users/patriciasilva/Desktop/temperatura2.py (3.7.3)
temp = int(input("Entre com a temperatura:"))

if temp < 5:
    print ("Congelando")
elif 0 <= temp < 10:
    print ("Frio")
elif 20 <= temp <= 25:
    print ("Normal")
elif 26 <= temp <= 35:
    print ("Quente")
else:
    print ("Muito Quente!")
```



```
Python 3.7.3 Shell
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar 25 2019, 16:39:00)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /Users/patriciasilva/Desktop/temperatura3.py =====
Entre com a temperatura:23
Normal
>>> |
```

Figura 17. Programação com uso dos comandos `if`/`else`/`elif`.

O comando `elif` permite que diversas condições possam ser inseridas; ao avaliar tais condições, o programa emite uma mensagem, de acordo com o valor avaliado (valor que é determinado pela condição). O comando condicional `if`, juntamente com os comandos aninhados, permitem que outras condições possam ser inseridas nas programações, fazendo com que possa ser seguido um caminho ou outro, dependendo da condição dada pelo programador.



Exemplo

Observe na Figura 18 o exemplo proposto por Lee (2011, p. 46), com o uso de números inteiros e comandos aninhados `if-else`:

```
testexyz.py - /Users/patriciasilva/Desktop/testexyz.py (3.7.3)
x = int(input("Por favor digite um número inteiro:"))
y = int(input("Por favor digite outro número inteiro:"))
z = int(input("Por favor digite um terceiro número inteiro:"))

if y>x:
    if z>y:
        print(z, "é maior")
    else:
        print(y, "é maior")
else:
    if z> x:
        print(z, "é maior")
    else:
        print(x, "é maior")

===== RESTART: /Users/patriciasilva/Desktop/testexyz.py =====
Por favor digite um número inteiro:4
Por favor digite outro número inteiro:2
Por favor digite um terceiro número inteiro:3
4 é maior
>>> |
```

Figura 18. Programação com uso dos comandos `if/else/elif`.

O exemplo da figura acima mostra que qualquer comando pode ser incluído dentro de comandos `if`. Quando várias condições são avaliadas, pode ser preciso colocar um comando `if` dentro do outro. Por meio de `x`, `y` e `z`, pode-se determinar qual número é maior, comparando, primeiramente, dois deles (`x` e `y`). Dependendo do resultado dessa condição, mais dois números são comparados. Ao aninhar as declarações, pode-se chegar à decisão de qual número é maior.



Referências

DOWNEY, A. B. *Pense em Python*: pense como um cientista da computação. São Paulo: Novatec, 2016. 312 p. Disponível em: <https://penseallen.github.io/PensePython2e/>. Acesso em: 23 jun. 2019.

LEE, K. D. *Python programming fundamentals*. New York; Dordrecht; Heidelberg; London: Springer, 2011. 241 p. (Undergraduate topics in Computer Science).

Leituras recomendadas

BORGES, L. E. *Python para desenvolvedores*. São Paulo: Novatec, 2014. 320 p.

CAELUM ENSINO E INOVAÇÃO. *Python e orientação a objetos*: curso PY-14. São Paulo: Caelum, 2019. p. 132–152. Disponível em: <https://www.caelum.com.br/apostila-python-orientacao-objetos/>. Acesso em: 23 jun. 2019.

MOREIRA DE MATOS, F. J. Exercício de Python: Recebe duas notas, calcula média e diz se foi aprovado. *Python Progressivo*, Fortaleza, fev. 2018. Disponível em: <https://www.pythonprogressivo.net/2018/02/Exercicio-Python-Recebe-Duas-Notas-Calcula-Media-Aprovado-Reprovado.html>. Acesso em: 23 jun. 2019.

SPAK, F. Python: Estrutura condicional `if-else`. *DevMedia*, Rio de Janeiro, 2017. Disponível em: <https://www.devmedia.com.br/python-estrutura-condicional-if-else/38217>. Acesso em: 23 jun. 2019.

SWEIGERT, A. *Invent your own computer games with Python*. 4. ed. San Francisco: No Starch Press, 2016. 376 p.

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO”. *Introdução à lógica de programação*. Guaratinguetá: Oficinas Tecnológicas para alunos do Ensino Médio – 2º encontro, 2011. 22 p. (Apostila). Disponível em: <http://www.feg.unesp.br/Home/Pesquisa23/inovee/oficinatecnologica/apostila---introducao-a-logica-de-programacao.compressed.pdf>. Acesso em: 23 jun. 2019.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS